

What and Why Java & Writing and Running Java

INTRODUCTION TO JAVA



Jim White
Java Developer

What is Java?

- General-purpose programming language
 - Used for cloud, server, web, desktop, mobile, and even AI applications
- Pervasive
 - Java is used extensively in finance, health, manufacturing, and many other industries,
 - Java can be found in over 3 billion mobile devices
- Java was used in the Mars Rover controller, Wikipedia Search, and Minecraft among other applications



¹ A) terenbro.com/blog/11-large-companies-that-use-java, B) blogs.oracle.com/javamagazine/post/the-top-25-greatest-java-apps-ever-written

Why learn Java?

- Straightforward to learn
 - An ideal place to start learning how to program
 - Large community providing an endless supply of resources for developers at any level
- Runs on any hardware architecture (x86,ARM, etc.) or operating system (Window, Linux, etc.)
- Robust, secure and open source
- Plenty of Java libraries for use in AI/ML/data engineering applications

Comments

- Use comments to leave human readable information in the code
 - Comments are ignored by compilers, tools, and the JRE
 - Comment lines are marked by `//`

```
// This is a Java comment - for humans to read but not for Java to process
```

- Because they are ignored, comments can go anywhere in the code

Java class

- Java code goes in a class
 - Define a class with the `class` keyword followed by the class name with curly brackets
`{ }`

```
// Java code goes into a class
class FirstJavaApp {

}
```

Methods

- Java applications consist of hundreds of classes
 - Methods are like sub-containers for code inside of classes
 - Each method accomplished a small, discrete bit of work
- Methods define reusable sets of code
 - Allowing the same code to be used with different inputs
 - Producing different outputs



¹ Image courtesy of Unsplash.com

Java method

- Java starts executing from the `main` method
- Define the main method with `main` and its own curly brackets `{ }`
 - `public static void` precedes `main`
 - `(String[] args)` after `main`

```
class FirstJavaApp {  
    public static void main(String[] args) {  
    }  
}
```

main = the starting point for an application
methods = sub-containers for code

Terminology cheat sheet

- class: a container for organizing Java code
- comment: human readable information in Java code that Java ignores
- method: like sub-containers for code inside of classes
- main method: the starting point or kick off place for a Java application

Review cheat sheet

```
// Code goes in a class ——— Java comment for humans to read but ignored by Java
class FirstJavaApp {
    }

```

class = a named container for all code

Let's practice!

INTRODUCTION TO JAVA

The Java Application

INTRODUCTION TO JAVA



Jim White
Java Developer

Java case sensitivity

- Java is case sensitive so `main` is not `MAIN` or `Main`

```
class FirstJavaApp {  
    public static void main(String[] args) {  
    }  
}
```

**not
MAIN or Main**

Java and camel case

- Java is case sensitive and uses *camel case*
- Camel case is the practice of writing phrases
 - Without spaces or punctuation
 - Capitalizing words in the phrase
- Example: The phrase "Hello everyone!" in camel case is **HelloEveryone**
 - Upper camel case - the first word starts in upper case
 - Lower camel case - the first word starts in lower case (**helloEveryone**)



¹ Image courtesy of free-images.com

Java naming

- Class names are **upper** camel case
 - Upper camel case is when the first word in the phrase is capitalized
 - If the class name phrase has only one word, then it is upper cased

```
FirstJavaApp  
WelcomeToJava  
Calculator
```

- Method names are **lower** camel case
 - Lower camel case is when the first word in the phrase is not capitalized
 - If the method name phrase has only one word, then it is lower cased

```
searchCustomers  
computeAverage  
main
```

println

- Use `System.out.println` with a string in `" "` to print something

```
class FirstJavaApp {  
    public static void main(String[] args) {  
        System.out.println("My first Java application");  
    }  
}
```



Use to output

- *println* is short for print line

Java statements

- Lines of code in methods are called *statements*.
 - All Java lines of code, i.e. statements, end in semicolon ;

```
class FirstJavaApp {  
    public static void main(String[] args) {  
        System.out.println("My first Java application");  
    }  
}
```


Whitespace

- Java ignores *whitespace*
 - Whitespace includes spaces, tabs or indentation, and empty lines
- Use whitespace to make your code more readable
 - Examples
 - Put empty or blank lines between the class and method
 - Indent methods in a class and statements in a method

```
class FirstJavaApp {  
    public static void main (String[] args) {  
        System.out.println("My first Java application");  
    }  
}
```

Blank lines between the class and methods

Indent methods

Indent statements in a method

More on comments

```
// A single line comment is denoted with two forward slashes
```

```
/* Multi-line comments are accomplished with forward slash and asterisk  
and then asterisk forward slash */
```

```
/**  
 * Use slash and two asterisks to start a documentation comment that will  
 * eventually be seen in HTML documentation produced by tools. Documentation  
 * comments end with slash asterisk like multi-line comments.  
 */
```

Let's practice!

INTRODUCTION TO JAVA

Variables and Primitives

INTRODUCTION TO JAVA



Jim White
Java Developer

What's a variable?

- Variables are named/labelled holders of data
 - Like labelled moving boxes
 - They are named to give an indication of the data they hold
- Variables are typed
 - Typing provides an indication of how the data can be operated on
 - Examples of types are numbers or characters



¹ Photo credit: Kadarius Seegars at unsplash.com

Variables

- Variable names should represent the data the variable holds (e.g., `bankBalance`)
 - Variable names are case sensitive
- Variables must be typed (integer, floating point, character, etc.)
- Variable data can change but its type may not
- Java is a "type safe" or "strongly typed" language

Types

- Typing is a way to classify information
 - Types determine the storage need for the data
 - Types determine allowed operations on the data
- The most basic types in Java are called *primitive* types
 - All other data types are constructed from primitive types
 - There are 8 primitive types

Primitive types

The 8 primitive types

- **byte** - integers between -128 and 128 (use 8 bits of memory)
- **short** - integers between -32,768 and 32,767 (use 16 bits of memory)
- **int** - integers between -2,147,483 and 2,147,647 (use 32 bits of memory)
- **long** - integers between -2^{63} and 2^{63} (use 64 bits of memory)
- **float** - for smaller floating/decimal point numbers (use 32 bits of memory)
- **double** - larger floating point numbers (use 64 bits of memory)
- **boolean** - either `true` or `false` values
 - Represent a yes/no, this/that, correct/incorrect or otherwise binary decision
- **char** - single letters, punctuation symbols, numbers, etc. indicated with single quotes `'`

Variable definition syntax

type name;

type name = value;

- Names are case sensitive
 - `age` is a different variable than `Age`
- Variables names typically follow *camel casing*
 - Uppercase the letters on word separation
 - Examples: `bankAccount` , `licensePlate`
 - Boolean names often start with `is`
 - Examples: `isActive` , `isMember`

```
// Age and age are different variables!  
int age;  
int Age;
```

```
// Assigning value when declaring  
double bankAccount = 245.38;
```

```
// boolean names start with is  
boolean isActive = true;
```

Declaring primitive variables

- Use `L` or `l` (big or little L) at the end of the number to indicate a **long** integer
- Use `F` or `f` (big or little F) to indicate a **float**.
- Use `D` or `d` (big or little D) to signal a **double**, but it is optional
- `true` and `false` are the two **boolean** values

```
long grossRevenue = 340000000L;  
long netRevenue  = 320000000l;
```

```
float unitPrice = 4.99f;  
float packagePrice = 11.55F;
```

```
double pi = 3.141592653589793;  
double lightSpeed = 1079252848.8D;
```

```
boolean isMinor = true;
```

Variable output

```
class UsingVars {  
    public static void main(String[] args) {  
        // Declaring customer age, shirt size, and sex using short, char, and boolean variables  
        short age = 35;  
        char shirtSize = 'M';  
        boolean isMale = true;  
        // Printing out the value of each of the 3 variables  
        System.out.println(age);  
        System.out.println(shirtSize);  
        System.out.println(isMale);  
    }  
}
```

```
35  
M  
true
```

Review Cheat Sheet

Primitive type	values	example
byte	-128 to 128	<code>byte age = 5;</code>
short	-32,768 to 32,767	<code>short sqrFeet = 3240;</code>
int	-2,147,483 and 2,147,647	<code>int accountBalance = 2500000;</code>
long	-2^{63} and 2^{63}	<code>long nationalDebt = 34849081491943L;</code>
float	$3.4e-038$ to $3.4e+038$	<code>float unitPrice = 4.99f</code>
double	$1.7e-308$ to $1.7e+308$	<code>double pi = 3.141592653589793;</code>
char	alphabet, punc, numbers, etc.	<code>char grade = 'C';</code>
boolean	<code>true</code> or <code>false</code>	<code>boolean isZoned = true;</code>

Primitive value cheat sheet

Integer and Floating Point Values

Primitive type	Value suffix	Example
byte	none	5
short	none	3240
int	none	2500000
long	l or L	34849081491943l or 34849081491943L
float	f or F	4.99f or 4.99F
double	optional d or D	3.1415 or 3.1415d or 3.1415D

Let's practice!

INTRODUCTION TO JAVA